

Tentu, mari kita bedah kode C# ini secara mendalam. Kode ini adalah bagian dari aplikasi pengolahan citra (seperti Photoshop sederhana) yang berfokus pada **Operasi Logika Biner** (Boolean Operations).

Secara garis besar, kode ini mengambil dua gambar, mengubahnya menjadi hitam-putih mutlak (biner), lalu membandingkan setiap pikselnya menggunakan logika matematika (AND, OR, XOR, NOT).

Berikut adalah penjelasan detail per bagian dan per baris kodenya.

1. Struktur Awal dan Pustaka

C#

```
using System;
using System.Drawing;

namespace MiniPhotoshop.Logic.ImageProcessing
{
    public static class LogicalOperations
    {
```

- **using System;**: Mengimpor tipe data dasar (seperti int, Math, Func).
- **using System.Drawing;**: Mengimpor pustaka grafis standar C# (GDI+), yang menyediakan kelas Bitmap dan Color untuk memanipulasi gambar.
- **namespace ...**: Mengelompokkan kode agar rapi dan tidak bentrok dengan nama kelas lain.
- **public static class LogicalOperations**: Kelas ini dideklarasikan static, artinya Anda tidak perlu membuat objek (new LogicalOperations()) untuk menggunakannya. Anda bisa langsung memanggil fungsinya seperti LogicalOperations.And(...).

2. Metode Pembantu Internal (Helper Methods)

Bagian ini berisi fungsi utilitas yang digunakan oleh fungsi utama, tetapi tidak bisa diakses dari luar kelas (private).

a. Konversi ke Biner (ToBinary)

C#

```
private static int ToBinary(Color c)
{
    int gray = (int)(c.R * 0.299 + c.G * 0.587 + c.B * 0.114);
    return (gray < 128) ? 0 : 255;
}
```

- **Tujuan:** Mengubah piksel berwarna menjadi angka biner: **0 (Hitam)** atau **255 (Putih)**.
- `int gray = ...`: Ini adalah rumus standar Luminositas untuk mengubah RGB menjadi Grayscale (abu-abu) agar sesuai dengan persepsi mata manusia. Rumus matematikanya adalah:

$$Y = 0.299R + 0.587G + 0.114B$$

Mata manusia lebih sensitif terhadap Hijau (G), jadi bobotnya paling besar (0.587).

- **(gray < 128) ? 0 : 255**: Ini disebut **Thresholding**.
 - Jika kecerahan di bawah 128 (gelap) $\rightarrow$ jadi 0 (Hitam pekat).
 - Jika kecerahan 128 ke atas (terang) $\rightarrow$ jadi 255 (Putih pekat).

b. Konversi Angka ke Warna (ToBinaryColor)

C#

```
private static Color ToBinaryColor(int val)
{
    return Color.FromArgb(val, val, val);
}
```

- **Tujuan:** Mengubah nilai angka (0 atau 255) kembali menjadi objek Color agar bisa digambar ke layar.
- **Color.FromArgb(val, val, val)**: Jika val adalah 0, hasilnya RGB(0,0,0) (Hitam). Jika 255, hasilnya RGB(255,255,255) (Putih).

3. Mesin Utama (PerformOperation)

Ini adalah fungsi paling kompleks dan cerdas dalam kode ini. Fungsi ini menangani perulangan piksel dan perbedaan ukuran gambar.

C#

```
private static Bitmap PerformOperation(Bitmap imgA, Bitmap imgB, Func<int, int, int> operation)
```

- **Parameter:** Menerima dua gambar (imgA, imgB) dan sebuah fungsi logika (operation).
Func<int, int, int> berarti fungsi ini menerima dua int (piksel A dan B) dan mengembalikan satu int (hasil).

C#

```
int newWidth = Math.Max(imgA.Width, imgB.Width);  
int newHeight = Math.Max(imgA.Height, imgB.Height);  
Bitmap resultBmp = new Bitmap(newWidth, newHeight);
```

- **Logika Ukuran:** Kode ini menyiapkan kanvas hasil (resultBmp) sebesar ukuran maksimum dari kedua gambar. Jika Gambar A ukuran 100x100 dan Gambar B 200x50, hasilnya akan 200x100 agar muat keduanya.

C#

```
for (int y = 0; y < newHeight; y++)  
{  
    for (int x = 0; x < newWidth; x++)  
    {
```

- **Looping:** Melakukan pemindaian piksel demi piksel dari kiri atas ke kanan bawah (koordinat x, y).

C#

```
bool inA = (x < imgA.Width && y < imgA.Height);  
bool inB = (x < imgB.Width && y < imgB.Height);
```

- **Pengecekan Batas:** Karena ukuran gambar mungkin berbeda, kita cek apakah koordinat (x, y) saat ini ada di dalam wilayah Gambar A (inA) atau Gambar B (inB). Ini mencegah error `IndexOutOfRangeException`.

C#

```
if (inA && inB)  
{  
    // KASUS 1: Piksel saling bertemu (Overlapping)  
    int valA = ToBinary(imgA.GetPixel(x, y));  
    int valB = ToBinary(imgB.GetPixel(x, y));  
    resultBmp.SetPixel(x, y, ToBinaryColor(operation(valA, valB)));  
}
```

- **Area Tumpang Tindih:** Jika piksel ada di kedua gambar, kita ambil nilai binernya, lalu jalankan fungsi operation (misalnya AND/OR), dan simpan hasilnya.

C#

```
else if (inA) { resultBmp.SetPixel(x, y, imgA.GetPixel(x, y)); }  
else if (inB) { resultBmp.SetPixel(x, y, imgB.GetPixel(x, y)); }  
else { resultBmp.SetPixel(x, y, Color.Black); }
```

- **Area Non-Overlapping:**
 - Jika hanya ada di A, salin piksel A mentah-mentah.
 - Jika hanya ada di B, salin piksel B mentah-mentah.
 - Jika tidak ada di keduanya (area kosong akibat perbesaran kanvas), isi dengan Hitam.

4. Metode Publik (API yang digunakan user)

Metode-metode ini menggunakan *Lambda Expression* (valA, valB) => ... untuk menentukan logika spesifik yang dikirim ke `PerformOperation`.

a. Logika AND

C#

```
public static Bitmap And(Bitmap imgA, Bitmap imgB)
{
    return PerformOperation(imgA, imgB, (valA, valB) =>
    {
        return (valA == 255 && valB == 255) ? 255 : 0;
    });
}
```

- **Konsep:** Hasilnya PUTIH (255) hanya jika **Gambar A Putih DAN Gambar B Putih**. Selain itu Hitam.
- **Visual:** Ini mencari irisan (intersection) dari bagian putih kedua gambar.

b. Logika OR

C#

```
public static Bitmap Or(Bitmap imgA, Bitmap imgB)
{
    return PerformOperation(imgA, imgB, (valA, valB) =>
    {
        return (valA == 255 || valB == 255) ? 255 : 0;
    });
}
```

- **Konsep:** Hasilnya PUTIH jika **Gambar A Putih ATAU Gambar B Putih** (atau keduanya).
- **Visual:** Ini menggabungkan (union) bentuk putih dari kedua gambar.

c. Logika XOR (Exclusive OR)

C#

```
public static Bitmap Xor(Bitmap imgA, Bitmap imgB)
{
    return PerformOperation(imgA, imgB, (valA, valB) =>
    {
        return (valA != valB) ? 255 : 0;
    });
}
```

- **Konsep:** Hasilnya PUTIH jika pikselnya **berbeda** (Satu putih, satu hitam). Jika sama (sama-sama putih atau sama-sama hitam), hasilnya Hitam.
- **Visual:** Ini berguna untuk mendeteksi perbedaan antara dua gambar.

5. Logika NOT (Invert)

C#

```
public static Bitmap Not(Bitmap imgA)
{
    // ... setup bitmap baru ...
    // ... looping x, y ...
    int valA = ToBinary(imgA.GetPixel(x, y));
    int valRes = (valA == 255) ? 0 : 255; // Balikkan nilai biner
    resultBmp.SetPixel(x, y, ToBinaryColor(valRes));
    // ... return ...
}
```

- **Perbedaan:** Metode ini tidak menggunakan PerformOperation karena hanya butuh satu gambar input (imgA).
- **Logika:** Membalik nilai.
 - Jika Putih (255) $\rightarrow$ jadi Hitam (0).
 - Jika Hitam (0) $\rightarrow$ jadi Putih (255).
- **Visual:** Seperti klise foto (negatif).

Ringkasan Alur Data

1. User memanggil LogicalOperations.And(gambar1, gambar2).
2. Fungsi PerformOperation membuat kanvas kosong seukuran max gambar.
3. Looping berjalan pixel demi pixel.
4. Warna piksel dikonversi ke **0 atau 255** (menggunakan rumus grayscale).

5. Logika AND diterapkan: $(255 \& 255) \rightarrow 255$, sisanya 0.
6. Hasil dikonversi kembali jadi warna dan disimpan ke gambar baru.
7. Gambar baru dikembalikan ke user.

Apakah Anda ingin saya memberikan contoh *unit test* sederhana untuk memverifikasi logika ini, atau mungkin menjelaskan bagaimana cara mengoptimalkan performa kode ini menggunakan LockBits?